

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2103

Course Title: Data Structures Lab

Assignment No: 03

Date of Submission: 4/01/2026

Submitted by,

Name: Durjoy Roy

Student ID: 00724105101055

Lab Section: B1

Academic Semester: Spring 2025

Program: BSc in CSE

Question:

1. Given an array of N integers, simulate Quick Sort using the last element as pivot and

count the number of comparisons performed during the sorting process.

2. Given an array and a pivot, rearrange the array such that elements less than pivot come

before it and greater come after.

3. Count the number of swaps required to sort an array using quick sort.

4. Output the pivot chosen at each recursive call in quick sort (always pick the last element as pivot).

5. For each recursive quick sort call, output the final index where pivot is placed.

6. For each pivot selected in quick sort (last element as pivot), count how many elements are less than the pivot at that step.

7. Given two sorted arrays, find the median of the combined sorted array using merge sort.

Also print the merged array.

8. Calculate the total number of comparisons made during merge sort.

9. Count the total number of merge operations performed when sorting an array using merge sort.

10. Determine the total number of recursive calls merge sort makes for a given array.

Solution 1:

```
#include <bits/stdc++.h>
```

```
using namespace std;
int comparisons = 0;
int partitions(vector<int>& arr, int st, int ed) {
    int pivot = arr[ed];
    int idx = st - 1;
    for(int j = st; j < ed; j++) {
        comparisons++;
        if(arr[j] <= pivot) {
            idx++;
            swap(arr[idx], arr[j]);
        }
    }
    idx++;
    swap(arr[idx], arr[ed]);
    return idx;
}
```

```
void quickSort(vector<int>& arr, int st, int ed) {
    if(st < ed) {
        int pi = partitions(arr, st, ed);
        quickSort(arr, st, pi - 1);
        quickSort(arr, pi + 1, ed);
    }
}
```

```

int main() {
    int N;
    cin >> N;
    vector<int> arr(N);
    for(int i = 0; i < N; i++) cin >> arr[i];

    quickSort(arr, 0, N - 1);

    cout << "Sorted Array: ";
    for(int x : arr) cout << x << " ";
    cout << "\nTotal Comparisons: " << comparisons << endl;
}

```

Output:

```

5
1 8 2 5 3
Sorted Array: 1 2 3 5 8
Total Comparisons: 6

```

Solution 2:

```

#include <bits/stdc++.h>
using namespace std;

```

```

int partPi(vector<int>& arr, int st, int ed)

```

```

{
    int pivot = arr[ed];
    int idx = st - 1;

    for(int j = st; j < ed; j++)
    {
        if(arr[j] < pivot)
        {
            idx++;
            swap(arr[j], arr[idx]);
        }
    }

    idx++;
    swap(arr[ed], arr[idx]);

    return idx;
}

```

```

void rearrangeArr(vector<int>& arr, int pivot)
{
    int n = arr.size();
    int pitIdx = -1;
    for(int i = 0; i < n; i++)

```

```
{
    if(arr[i] == pivot)
    {
        pitIdx = i;
        break;
    }
}
swap(arr[pitIdx], arr[n - 1]);
partPi(arr, 0, n - 1);
}
```

```
int main()
{
    int n;
    cin >> n;

    vector<int> arr(n);
    for(int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    int pivot;
    cin >> pivot;
```

```

    rearrangeArr(arr,pivot);
    for(int x : arr)
        cout << x << " ";
    cout << endl;

    return 0;
}

```

Output:

```

5
3 7 2 5 1
5
3 2 1 5 7

```

Solution 3:

```

#include <bits/stdc++.h>
using namespace std;

int swapCount = 0;

int partitions(vector<int>& arr, int st, int ed) {
    int pivot = arr[ed];
    int idx = st - 1;

```

```

for(int j = st; j < ed; j++) {
    if(arr[j] <= pivot) {
        idx++;
        swap(arr[idx], arr[j]);
        swapCount++;
    }
}
idx++;
swap(arr[idx], arr[ed]);
swapCount++;
return idx;
}

```

```

void quickSort(vector<int>& arr, int st, int ed) {
    if(st < ed) {
        int pi = partitions(arr, st, ed);
        quickSort(arr, st, pi - 1);
        quickSort(arr, pi + 1, ed);
    }
}

```

```

int main() {
    int N;
    cin >> N;

```



```
vector<int> arr(N);  
for(int i = 0; i < N; i++) cin >> arr[i];  
  
quickSort(arr, 0, N - 1);  
  
cout << "Total Swaps: " << swapCount << endl;  
}
```

Output:

```
5  
4 2 7 5 6  
Total Swaps: 8
```

Solution 4:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int partions(vector<int>& arr, int st, int ed) {  
    int pivot = arr[ed];  
    int idx = st - 1;  
  
    for(int j = st; j < ed; j++) {  
        if(arr[j] <= pivot) {  
            idx++;  
        }  
    }  
}
```

```
        swap(arr[idx], arr[j]);  
    }  
}
```

```
    idx++;  
    swap(arr[idx], arr[ed]);  
    return idx;  
}
```

```
void quickSort(vector<int>& arr, int st, int ed) {  
    if(st < ed) {  
  
        cout << arr[ed] << " ";  
  
        int pi = partions(arr, st, ed);  
        quickSort(arr, st, pi - 1);  
        quickSort(arr, pi + 1, ed);  
    }  
}
```

```
int main() {  
    int n;  
    cin >> n;
```

```
vector<int> arr(n);  
for(int i = 0; i < n; i++)  
    cin >> arr[i];  
  
quickSort(arr, 0, n - 1);  
  
return 0;  
}
```

Output:

```
5  
4 2 7 5 6  
6 5 2
```

Solution 5:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int partions(vector<int>& arr, int st, int ed)  
{  
    int pivot = arr[ed];  
    int idx = st - 1;  
  
    for(int j = st; j < ed; j++)
```

```

{
    if(arr[j] <= pivot)
    {
        idx++;
        swap(arr[idx], arr[j]);
    }
}

idx++;
swap(arr[idx], arr[ed]);
return idx;
}

void quickSort(vector<int>& arr, int st, int ed)
{
    if(st < ed)
    {
        int pi = partions(arr, st, ed);
        cout << pi << " ";

        quickSort(arr, st, pi - 1);
        quickSort(arr, pi + 1, ed);
    }
}

```

```
int main()
{
    int n;
    cin >> n;

    vector<int> arr(n);
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);

    return 0;
}
```

Output:

```
5
4 2 5 1 3
2 0 4
```

Solution 6:

```
#include <bits/stdc++.h>
using namespace std;
```

```
int partions(vector<int>& arr, int st, int ed)
{
    int pivot = arr[ed];
    int idx = st - 1;
    int c = 0;

    for(int j = st; j < ed; j++)
    {
        if(arr[j] <= pivot)
        {
            idx++;
            swap(arr[idx], arr[j]);
            c++;
        }
    }

    idx++;
    swap(arr[idx], arr[ed]);

    cout << "Pivot: " << pivot << " less than ele: " << c << endl;
    return idx;
}
```

```
void quickSort(vector<int>& arr, int st, int ed) {  
    if(st < ed) {  
        int pi = partions(arr, st, ed);  
        quickSort(arr, st, pi - 1);  
        quickSort(arr, pi + 1, ed);  
    }  
}
```

```
int main(){  
    int n;  
    cin >> n;  
  
    vector<int> arr(n);  
    for(int i = 0; i < n; i++)  
        cin >> arr[i];  
  
    quickSort(arr, 0, n - 1);  
  
}
```

Output:

5

3 8 2 5 1

Pivot: 1 less than ele: 0

Pivot: 3 less than ele: 1

Pivot: 8 less than ele: 1

Solution 7:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void merage(vector<int>&arr,int st,int mid,int ed)
```

```
{
```

```
    vector<int>temp;
```

```
    int i=st;
```

```
    int j=mid+1;
```

```
    while(i<=mid && j<=ed)
```

```
    {
```

```
        if(arr[i]<=arr[j])
```

```
        {
```

```
            temp.push_back(arr[i++]);
```

```
        }
```

```
        else
```

```
        {
```

```
            temp.push_back(arr[j++]);
```

```
        }
```

```
    }
```

```
    while(i<=mid) temp.push_back(arr[i++]);
```



```
while(j<=ed) temp.push_back(arr[j++]);
```

```
for(int idx=0; idx<temp.size(); idx++)  
{  
    arr[st+idx]=temp[idx];  
}  
}
```

```
void margeSort(vector<int>&arr,int st, int ed)  
{  
    if(st<ed)  
    {  
        int mid = st + (ed-st)/2;  
        margeSort(arr,st,mid);  
        margeSort(arr,mid+1,ed);  
  
        merage(arr,st,mid,ed);  
    }  
}
```

```
double findMedian(vector<int>&arr){  
    int s = arr.size();
```

```
    if(s % 2 == 1) {  
        return arr[s/2];  
    } else {  
        return (arr[s/2 - 1] + arr[s/2]) / 2.0;  
    }  
}
```

```
int main()  
{  
    int n, m;  
    cout << "first array size: ";  
    cin>>n;  
  
    vector<int> A(n);  
    for(int i=0; i<n; i++) cin>>A[i];  
  
    cout << "Second array size: ";  
    cin>>m;  
  
    vector<int> B(m);  
    for(int i=0; i<m; i++) cin>>B[i];  
  
    vector<int> merged;  
    for(int x : A) merged.push_back(x);
```

```

for(int x : B) merged.push_back(x);

margeSort(merged, 0, merged.size()-1);
for(int x : merged) cout << x << " ";
cout << endl;

double ans = findMedian(merged);
cout << "Median: " << ans << endl;

return 0;
}

```

Output:

```

first array size: 5
1 3 5 7 9
Second array size: 4
2 4 6 8
1 2 3 4 5 6 7 8 9
Median: 5

```

Solution 8:

```

#include <bits/stdc++.h>
using namespace std;

```

```
int compCount = 0;
```

```
void merage(vector<int>&arr,int st,int mid,int ed) {
```

```
    vector<int>temp;
```

```
    int i=st;
```

```
    int j=mid+1;
```

```
    while(i<=mid && j<=ed) {
```

```
        compCount++;
```

```
        if(arr[i]<=arr[j]) {
```

```
            temp.push_back(arr[i]);
```

```
            i++;
```

```
        } else {
```

```
            temp.push_back(arr[j]);
```

```
            j++;
```

```
        }
```

```
    }
```

```
    while(i<=mid) {
```

```
        temp.push_back(arr[i]);
```

```
        i++;
```

```
    }
```

```
    while(j<=ed) {
```

```
        temp.push_back(arr[j]);
```

```

        j++;
    }

    for(int idx=0;idx<temp.size();idx++){
        arr[st+idx]=temp[idx];
    }
}

void margearr(vector<int>&arr,int st, int ed) {
    if(st<ed) {
        int mid = st + (ed-st)/2;
        margearr(arr,st,mid);
        margearr(arr,mid+1,ed);

        merage(arr,st,mid,ed);
    }
}

int main() {
    int n;
    cin>>n;
    vector<int>arr(n);
    for(int &p:arr) cin>>p;

```

```

    margearr(arr,0,arr.size()-1);
    for(int val:arr){
        cout << val << " ";
    }
    cout << "\nTotal comparisons: " << compCount << endl;

    return 0;
}

```

Output:

```

5
5 3 2 4 1
1 2 3 4 5
Total comparisons: 7

```

Solution 9:

```

#include <bits/stdc++.h>
using namespace std;

int mergeOp = 0;

void merage(vector<int>&arr,int st,int mid,int ed) {
    mergeOp++;
    vector<int>temp;

```

```
int i=st;
int j=mid+1;
while(i<=mid && j<=ed) {
    if(arr[i]<=arr[j]) {
        temp.push_back(arr[i]);
        i++;
    } else {
        temp.push_back(arr[j]);
        j++;
    }
}
```

```
while(i<=mid) {
    temp.push_back(arr[i]);
    i++;
}
```

```
while(j<=ed) {
    temp.push_back(arr[j]);
    j++;
}
```

```
for(int idx=0;idx<temp.size();idx++){
    arr[st+idx]=temp[idx];
}
```

```
}  
}
```

```
void margearr(vector<int>&arr,int st, int ed) {  
    if(st<ed) {  
        int mid = st + (ed-st)/2;  
        margearr(arr,st,mid);  
        margearr(arr,mid+1,ed);  
  
        merage(arr,st,mid,ed);  
    }  
}
```

```
int main() {  
    int n;  
    cin>>n;  
    vector<int>arr(n);  
    for(int &p:arr) cin>>p;  
  
    margearr(arr,0,arr.size()-1);  
    for(int val:arr){  
        cout << val << " ";  
    }  
}
```



```
cout << "\nTotal merge operations: " << mergeOp << endl;
```

```
return 0;
```

```
}
```

Output:

5

5 3 2 4 1

1 2 3 4 5

Total merge operations: 4

Solution 10:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int recursiveCalls = 0;
```

```
void merage(vector<int>&arr,int st,int mid,int ed) {
```

```
    vector<int>temp;
```

```
    int i=st;
```

```
    int j=mid+1;
```

```
    while(i<=mid && j<=ed) {
```

```
        if(arr[i]<=arr[j]) {
```

```
            temp.push_back(arr[i]);
```

```
            i++;
```

```

    } else {
        temp.push_back(arr[j]);
        j++;
    }
}

while(i<=mid) {
    temp.push_back(arr[i]);
    i++;
}

while(j<=ed) {
    temp.push_back(arr[j]);
    j++;
}

for(int idx=0;idx<temp.size();idx++){
    arr[st+idx]=temp[idx];
}
}

void margearr(vector<int>&arr,int st, int ed) {
    recursiveCalls++;

```

```

if(st<ed) {
    int mid = st + (ed-st)/2;
    margearr(arr,st,mid);
    margearr(arr,mid+1,ed);

    merage(arr,st,mid,ed);
}
}

```

```

int main() {
    int n;
    cin>>n;
    vector<int>arr(n);
    for(int &p:arr) cin>>p;

    margearr(arr,0,arr.size()-1);
    for(int val:arr){
        cout << val << " ";
    }
    cout << "\nTotal recursive calls: " << recursiveCalls << endl;

    return 0;
}

```

Output:

5

5 3 2 4 1

1 2 3 4 5

Total recursive calls: 9